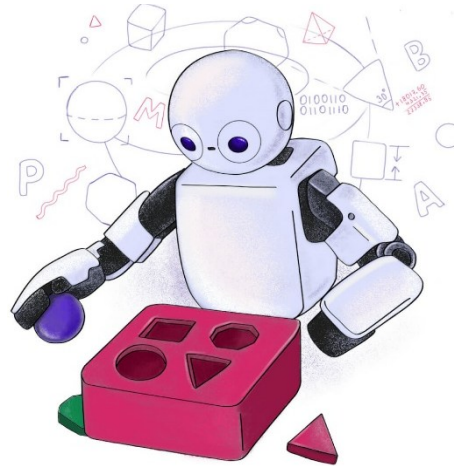


# TP558 - Tópicos avançados em Machine Learning: **AutoGluon-Tabular**



***Inatel***

Bryan Fernando Sarango Rodriguez  
[bryan.rodriguez@dtel.inatel.br](mailto:bryan.rodriguez@dtel.inatel.br)

# AutoGluon Tabular é um framework de AutoML (Machine Learning Automatizado)

## 1 ML, na prática

pegar dados, escolher um algoritmo, ajustá-lo e avaliar se ele acerta em dados novos

## 2 Decisão humana

limpar os dados, criar atributos, selecionar modelos, validar. Errar em qualquer um deles compromete o resultado.

## 3 Sucesso do ML

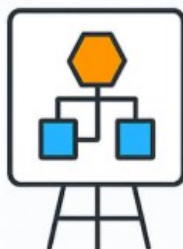
Mais aplicações (saúde, finanças, indústria, telecom) e mais técnicas, tornou a uma pessoa difícil de dominar

## 4 Solução

AutoML recebe os dados, define como modelar e entrega um modelo pronto. O AutoGluon faz isso



Data  
collection  
and  
preparation



Feature  
engineering  
and  
selection



Model  
selection and  
hyperparameter  
tuning



Model  
evaluation  
and  
validation



Model  
deployment  
and monitoring

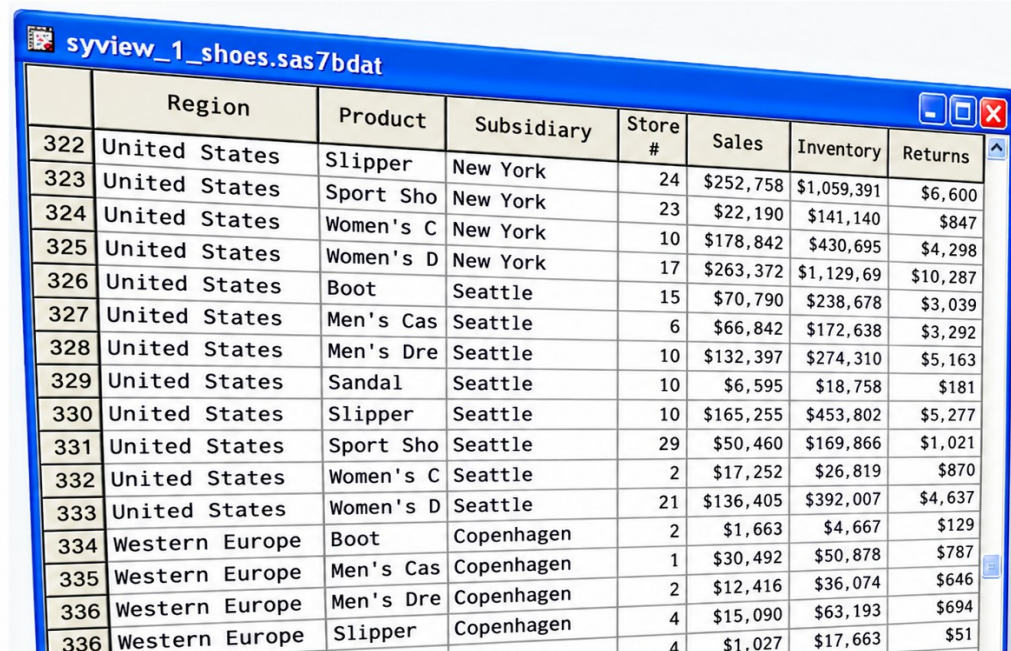
Etapas do pipeline de ML que o AutoML se propõe a automatizar — da coleta de dados ao deploy.

# Dados tabulares: o formato mais comum e o mais heterogêneo

Linhas independentes; colunas numéricas, categóricas, texto, datas, IDs, valores ausentes — tudo na mesma tabela.

Sem estrutura espacial ou sequencial: CNNs e RNNs não se aplicam; árvores dominam.

Muitos frameworks exigem dados já limpos e numéricos — devolvem ao humano a parte mais tediosa.



|     | Region         | Product   | Subsidiary | Store # | Sales     | Inventory   | Returns  |
|-----|----------------|-----------|------------|---------|-----------|-------------|----------|
| 322 | United States  | Slipper   | New York   | 24      | \$252,758 | \$1,059,391 | \$6,600  |
| 323 | United States  | Sport Sho | New York   | 23      | \$22,190  | \$141,140   | \$847    |
| 324 | United States  | Women's C | New York   | 10      | \$178,842 | \$430,695   | \$4,298  |
| 325 | United States  | Women's D | New York   | 17      | \$263,372 | \$1,129,69  | \$10,287 |
| 326 | United States  | Boot      | Seattle    | 15      | \$70,790  | \$238,678   | \$3,039  |
| 327 | United States  | Men's Cas | Seattle    | 6       | \$66,842  | \$172,638   | \$3,292  |
| 328 | United States  | Men's Dre | Seattle    | 10      | \$132,397 | \$274,310   | \$5,163  |
| 329 | United States  | Sandal    | Seattle    | 10      | \$6,595   | \$18,758    | \$181    |
| 330 | United States  | Slipper   | Seattle    | 10      | \$165,255 | \$453,802   | \$5,277  |
| 331 | United States  | Sport Sho | Seattle    | 29      | \$50,460  | \$169,866   | \$1,021  |
| 332 | United States  | Women's C | Seattle    | 2       | \$17,252  | \$26,819    | \$870    |
| 333 | United States  | Women's D | Seattle    | 21      | \$136,405 | \$392,007   | \$4,637  |
| 334 | Western Europe | Boot      | Copenhagen | 2       | \$1,663   | \$4,667     | \$129    |
| 335 | Western Europe | Men's Cas | Copenhagen | 1       | \$30,492  | \$50,878    | \$787    |
| 336 | Western Europe | Men's Dre | Copenhagen | 2       | \$12,416  | \$36,074    | \$646    |
| 336 | Western Europe | Men's Dre | Copenhagen | 4       | \$15,090  | \$63,193    | \$694    |
| 336 | Western Europe | Slipper   | Copenhagen | 4       | \$1,027   | \$17,663    | \$51     |

```
>>> from autogluon import TabularPrediction as task
>>> predictor = task.fit(train_data=task.Dataset(file_path=TRAIN_DATA.csv), label_column=COLUMN_NAME)
>>> predictions = predictor.predict(task.Dataset(file_path=TEST_DATA.csv))
```

Uma tabela real: tipos de coluna misturados. Abaixo, as três linhas com que o AutoGluon treina e prevê a partir dela.

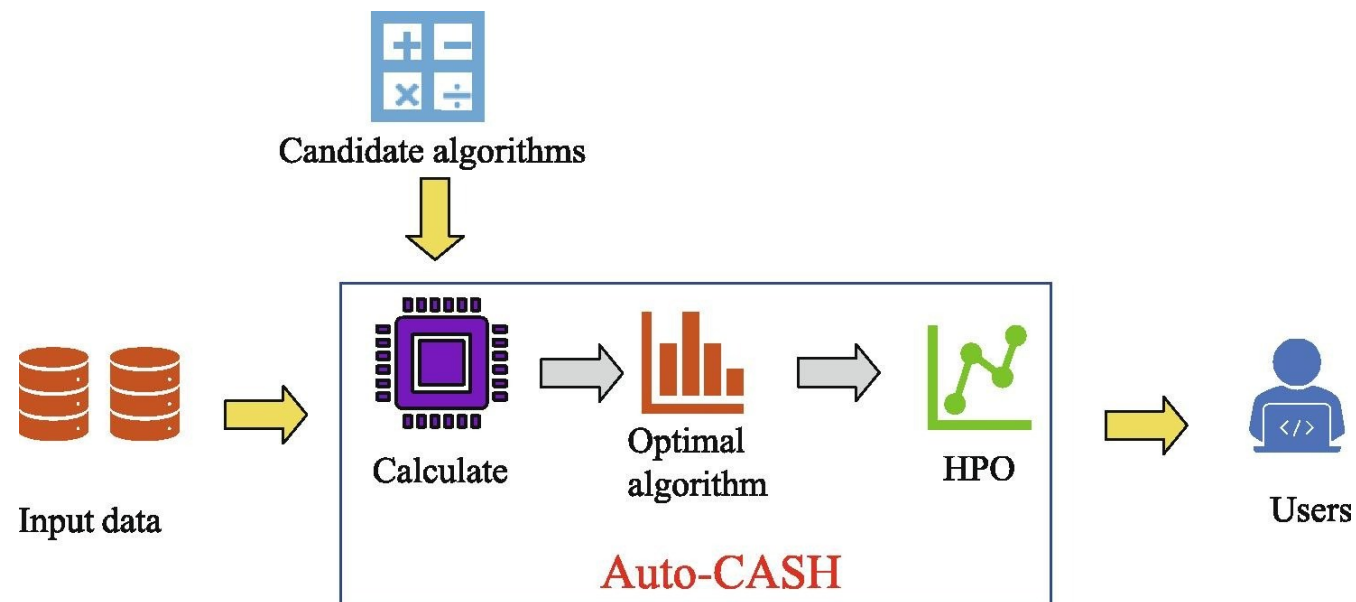
# AutoML virou sinônimo de CASH — e isso é uma limitação

AutoML automatiza as etapas do machine learning: preparação de dados, escolha de algoritmo, ajuste de hiperparâmetros e combinação de modelos

CASH = escolher algoritmo + hiperparâmetros por busca (Auto-WEKA - bayesiana, TPOT- genética, aleatória).

Busca intratável: gasta horas em configurações que nenhum especialista testaria.

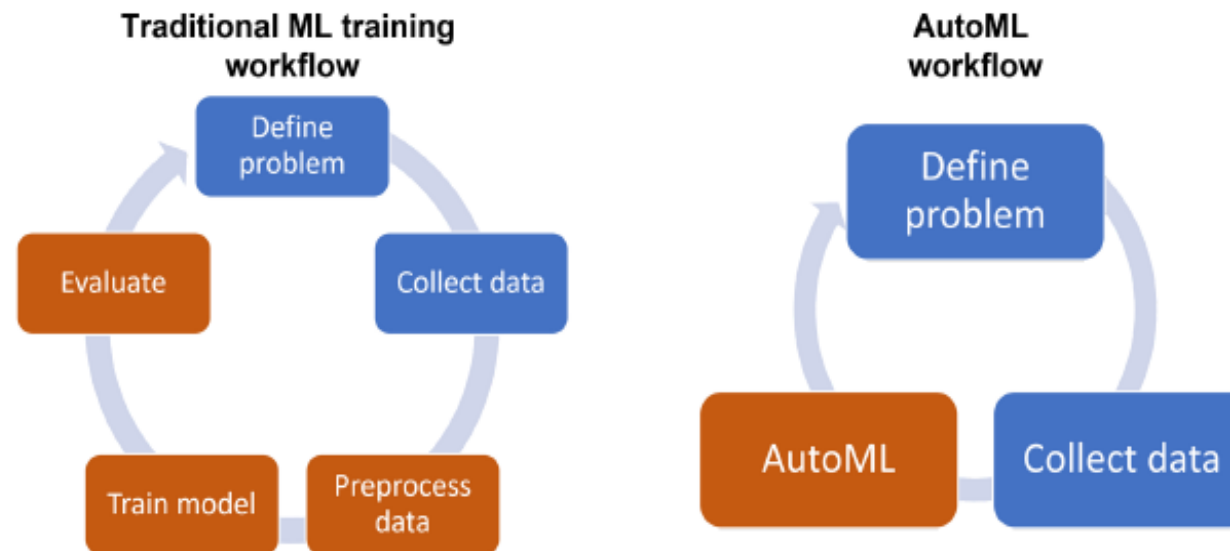
Aposta do AutoGluon: CASH é só uma peça. O tempo rende mais em pré-processamento, modelos diversos e ensembles.



O paradigma Auto-CASH: dados + algoritmos candidatos → busca do melhor algoritmo → otimização de hiperparâmetros.

# O que é o AutoGluon-Tabular

- 1 Biblioteca Python open-source (Amazon, 2020) para classificação e regressão em tabelas.
- 2 Uma linha de código: CSV cru entra, preditor pronto sai; o tipo de problema é inferido.
- 3 Quatro princípios: simplicidade, robustez, tolerância a falhas e tempo previsível.
- 4 Primeiro framework a codificar, em um só sistema, as práticas da comunidade Kaggle.



O ciclo manual (definir, coletar, pré-processar, treinar, avaliar) colapsa em um único passo: AutoML.

# Onde o AutoGluon se afasta dos outros frameworks

| Framework                | Dados crus? | Estratégia CASH                | Ensembling                                     |
|--------------------------|-------------|--------------------------------|------------------------------------------------|
| Auto-WEKA (2013)         | Não         | Otimização bayesiana           | Bagging, boosting, stacking, voto              |
| auto-sklearn (2015)      | Não         | BayesOpt + meta-learning       | Ensemble selection                             |
| TPOT                     | Não         | Programação genética           | Stacking                                       |
| H2O AutoML               | Sim         | Busca aleatória                | Stacking + bagging                             |
| GCP AutoML Tables        | Sim         | Não divulgada                  | Boosting (AdaNet)                              |
| <b>AutoGluon-Tabular</b> | <b>Sim</b>  | <b>Nenhuma (padrões fixos)</b> | <b>Multi-layer stacking + repeated bagging</b> |

# Pré-processamento automático

Numérica

Categórica

Texto → n-gramas

Data → numérico

ID único → descartada

Tipagem automática de cada coluna no estágio agnóstico ao modelo.

Tipo de problema inferido da coluna-alvo (texto → classificação; numérico → regressão).

Dois estágios: um agnóstico para todos os modelos, outro específico por modelo (árvores ≠ rede neural).

Faltante categórico vira “Unknown”: não se imputa, porque a ausência carrega informação.

Categoria nunca vista no teste cai em “Unknown” — sem quebrar o modelo.

**Por que importa: dados raramente faltam ao acaso — a ausência é evidência, e a categoria Unknown a preserva.**

## Com os dados prontos, quais modelos o AutoGluon treina

Random Forest e Extra-Trees → LightGBM e CatBoost →  
rede neural própria → kNN.

Diversidade é o objetivo: árvores (fronteiras nos eixos) +  
rede (fronteiras suaves) + kNN (local).

Ordem deliberada: confiáveis primeiro, caros e instáveis  
por último — resultado garantido com pouco tempo.

Hiperparâmetros padrão, definidos a priori. Sem busca.

### COMPARING POPULAR MACHINE LEARNING MODELS



Famílias de modelos base: árvores isoladas, florestas (bagging) e boosting (aqui XGBoost; o AutoGluon usa LightGBM e CatBoost).

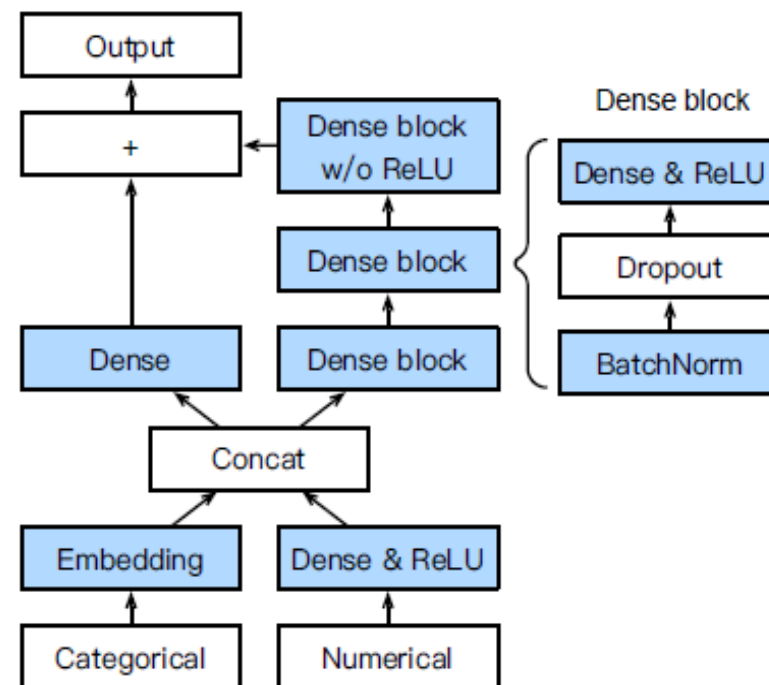
# A rede neural: embeddings por variável e um atalho linear

Cada categórica ( $\geq 4$  níveis) ganha um embedding próprio — vetor denso no lugar do one-hot.

Concatenação → 3 blocos densos (Dense+ReLU, Dropout, BatchNorm).

Skip connection: atalho linear da concatenação direto à saída — melhor gradiente, melhores embeddings.

Raramente é o melhor modelo isolado; é o mais valioso para o ensemble.



*Figure 1.* Architecture of AutoGluon's neural network for tabular data composed of numerical and categorical features. Layers with learnable parameters are marked as blue.

# Três formas de combinar modelos

## Bagging

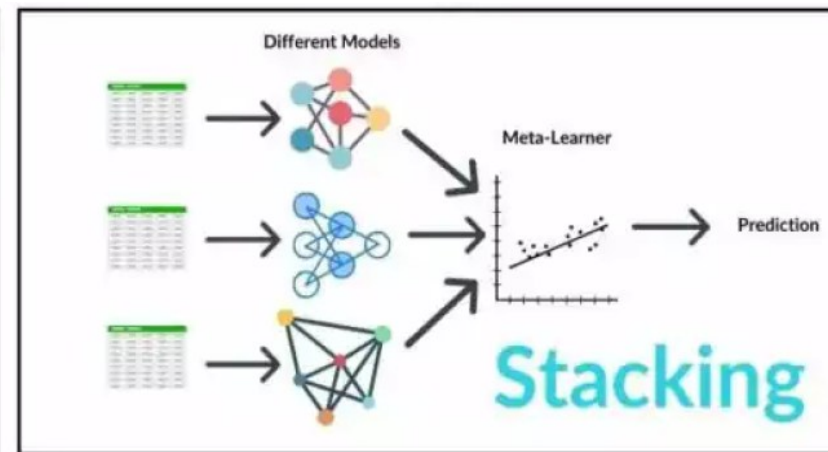
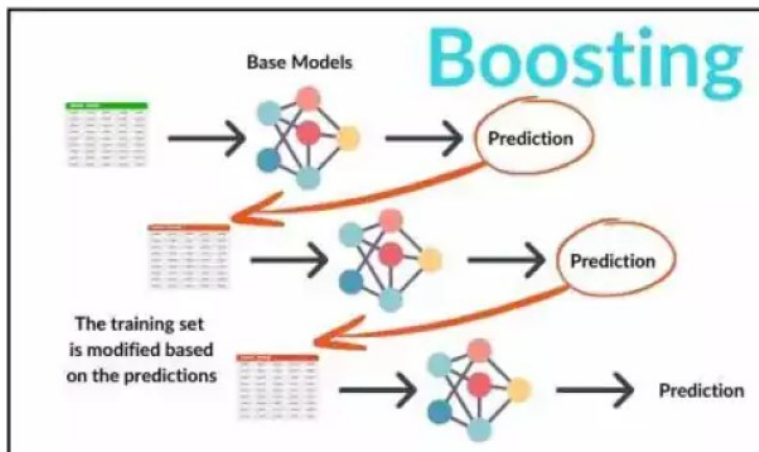
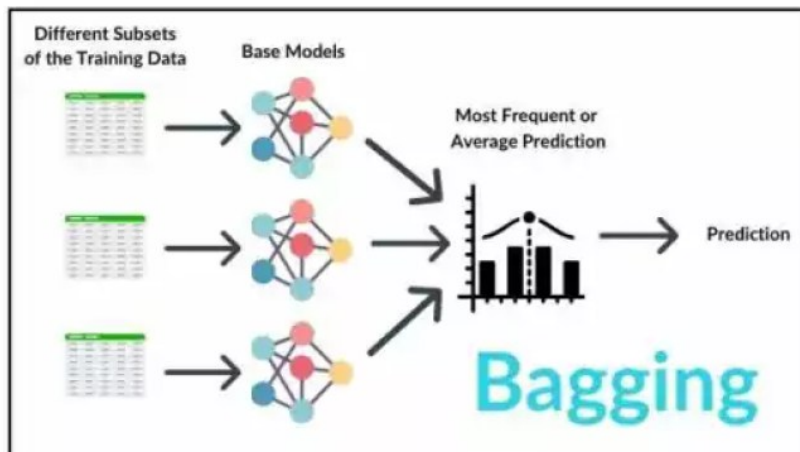
Vários subconjuntos, um modelo em cada, média das saídas. Reduz variância. Ex.: Random Forest.

## Boosting

Modelos em sequência; cada um corrige os erros do anterior. Reduz viés. Ex.: LightGBM, CatBoost.

## Stacking

Predições dos modelos base viram entradas de um meta-modelo que aprende a combiná-las.



## A arquitetura de ensemble Multi-layer stacking

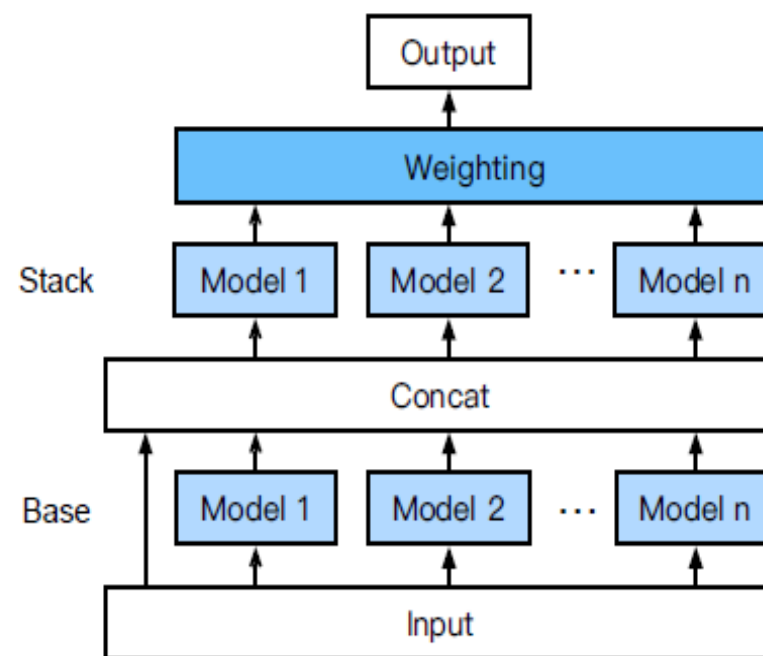
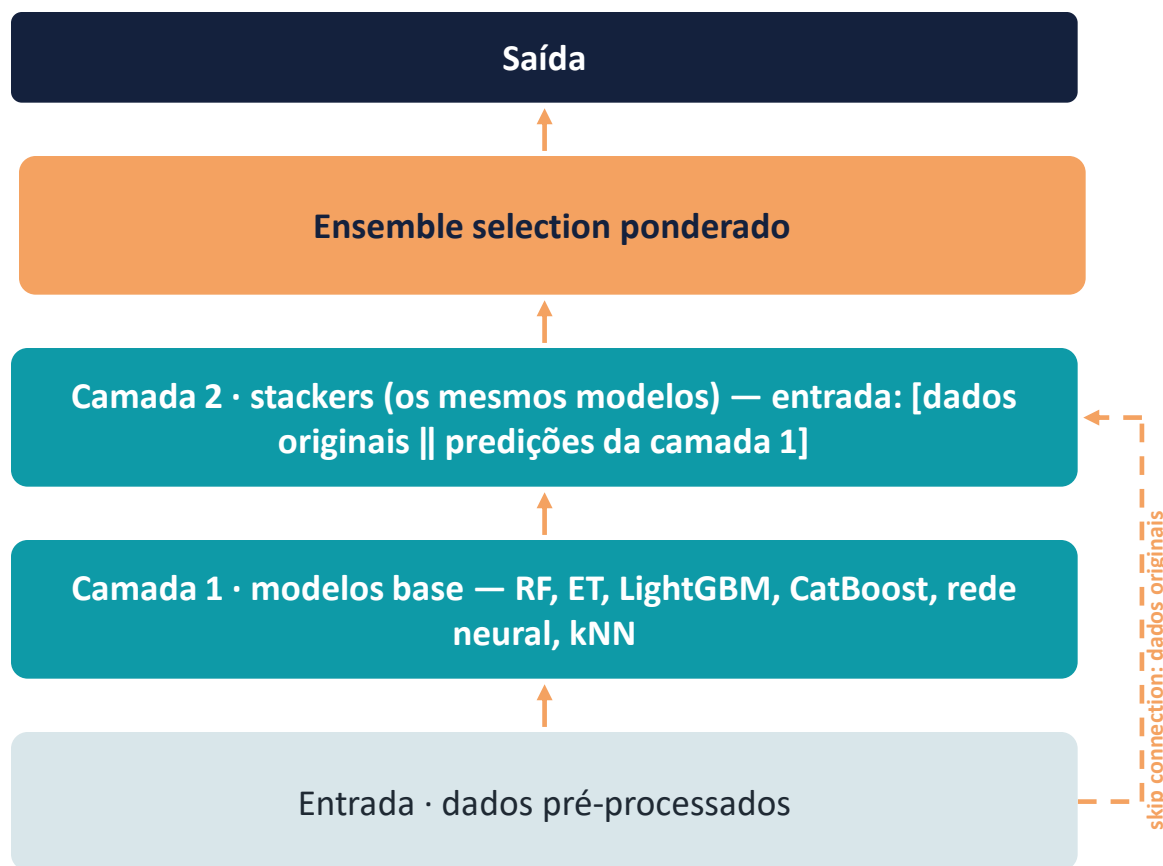


Figure 2. AutoGluon's multi-layer stacking strategy, shown here using two stacking layers and  $n$  types of base learners.

## Predições out-of-fold: stacking sem vazamento de dados

|         | Dobra 1 | Dobra 2 | Dobra 3 | Dobra 4 | Dobra 5 |
|---------|---------|---------|---------|---------|---------|
| Cópia 1 | prevê   | treina  | treina  | treina  | treina  |
| Cópia 2 | treina  | prevê   | treina  | treina  | treina  |
| Cópia 3 | treina  | treina  | prevê   | treina  | treina  |
| Cópia 4 | treina  | treina  | treina  | prevê   | treina  |
| Cópia 5 | treina  | treina  | treina  | treina  | prevê   |

Cada cópia treina em 4 dobras e prevê a que não viu (laranja). Empilhando as células laranja, há uma predição OOF para 100% do treino.

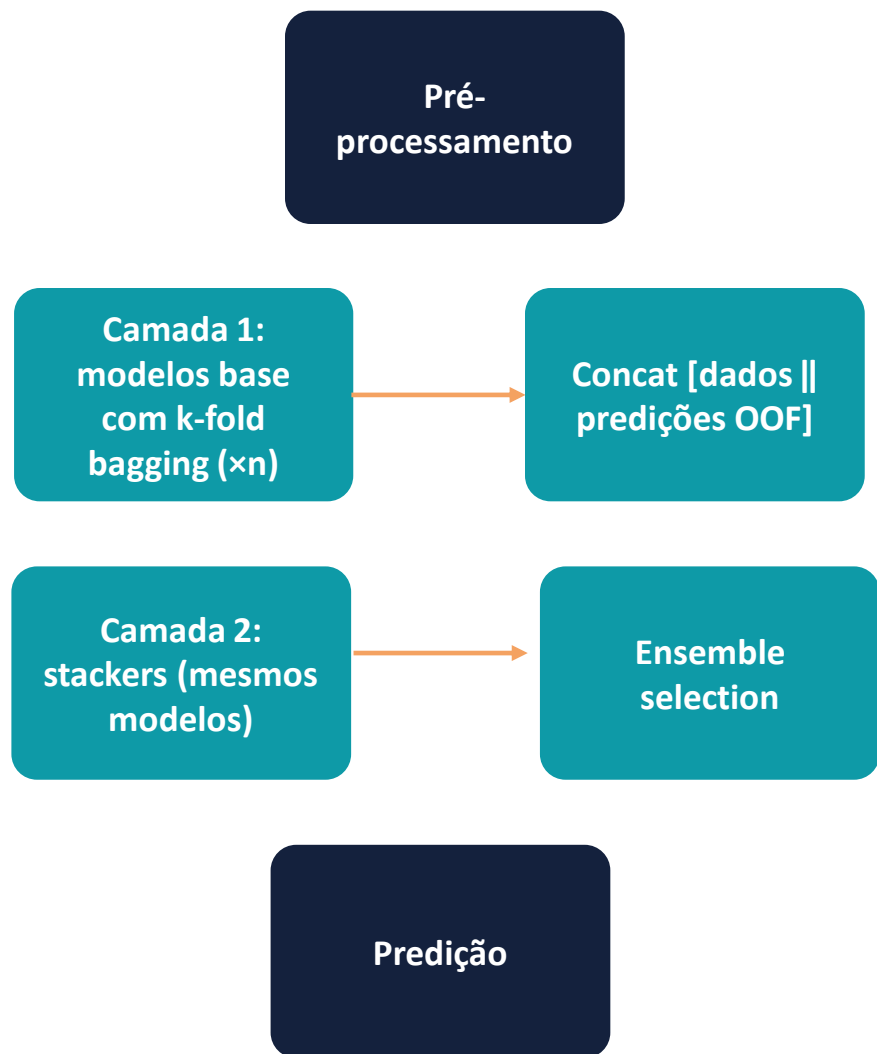
O uso de predições sobre os próprios dados de treino torna o stacker excessivamente confiante e provoca vazamento de dados.

O AutoGluon aplica k-fold bagging para que cada cópia do modelo preveja apenas a dobra que não utilizou no treinamento.

As predições out-of-fold permitem que o stacker aprenda com predições honestas para 100% dos exemplos de treino.

O treinamento de cada modelo kk vezes aumenta o custo computacional, mas permite empilhar várias camadas sem vazamento de dados.

## O pipeline dentro de fit(): camadas que geram previsões honestas




---

**Algorithm 1** AutoGluon-Tabular Training Strategy  
(multi-layer stack ensembling +  $n$ -repeated  $k$ -fold bagging).

---

**Require:** data  $(X, Y)$ , family of models  $\mathcal{M}$ , # of layers  $L$

```

1: Preprocess data to extract features
2: for  $l = 1$  to  $L$  do {Stacking}
3:   for  $i = 1$  to  $n$  do { $n$ -repeated}
4:     Randomly split data into  $k$  chunks  $\{X^j, Y^j\}_{j=1}^k$ 
5:     for  $j = 1$  to  $k$  do { $k$ -fold bagging}
6:       for each model type  $m$  in  $\mathcal{M}$  do
7:         Train a type- $m$  model on  $X^{-j}, Y^{-j}$ 
8:         Make predictions  $\hat{Y}_{m,i}^j$  on OOF data  $X^j$ 
9:       end for
10:    end for
11:  end for
12:  Average OOF predictions  $\hat{Y}_m \leftarrow \{\frac{1}{n} \sum_i \hat{Y}_{m,i}^j\}_{j=1}^k$ 
13:   $X \leftarrow \text{concatenate}(X, \{\hat{Y}_m\}_{m \in \mathcal{M}})$ 
14: end for
  
```

---

## Como o AutoGluon foi avaliado

6

frameworks AutoML

50

datasets (39 OpenML + 11 Kaggle)

1 h / 4 h

limites de tempo, mesma instância EC2

0

hiperparâmetros otimizados no AutoGluon

- 1 OpenML: 39 datasets de classificação, 10 divisões cada; métricas 1-AUC e log-loss.
- 2 Kaggle: 11 competições reais; métrica = percentil no leaderboard (fração de equipes humanas superadas).
- 3 Todas as configurações foram padrão — reforçando que o AutoGluon rodou sem otimizar um único hiperparâmetro

# OpenML: campeão em 23 de 39 — mais que todos os outros somados

| Framework    | Falhas | Campeão | Rank médio | Perda reescalada | Tempo (min) |
|--------------|--------|---------|------------|------------------|-------------|
| AutoGluon    | 1      | 23      | 1,84       | 0,139            | 201         |
| H2O AutoML   | 8      | 2       | 3,13       | 0,245            | 220         |
| TPOT         | 5      | 5       | 3,38       | 0,203            | 235         |
| GCP-Tables   | 14     | 4       | 3,75       | 0,334            | 195         |
| auto-sklearn | 6      | 3       | 3,81       | 0,320            | 240         |
| Auto-WEKA    | 6      | 1       | 5,09       | 0,800            | 244         |

- 1 Único com rank médio abaixo de 2; perda quase metade do segundo colocado.
- 2 Uma falha contra 5 a 14 — e 201 min dentro do limite de 240, que vários rivais estouram.

# Kaggle: top 1% contra equipes humanas, a partir do CSV cru

| Framework    | Falhas | Campeão | Rank médio | Percentil médio |
|--------------|--------|---------|------------|-----------------|
| AutoGluon    | 0      | 7       | 1,71       | 0,704           |
| GCP-Tables   | 1      | 3       | 2,29       | 0,628           |
| H2O AutoML   | 3      | 0       | 3,43       | 0,513           |
| TPOT         | 1      | 0       | 3,71       | 0,471           |
| auto-sklearn | 0      | 1       | 3,86       | 0,482           |
| Auto-WEKA    | 1      | 0       | 6,00       | 0,206           |

Percentil = fração de equipes humanas superadas no leaderboard oficial (maior é melhor).

O rival mais próximo (GCP) usa 92 servidores. Equipes humanas levam semanas; aqui, 4 h sem intervenção.

7 / 11

competições em que foi campeão

70 %

das equipes humanas superadas (4 h)

99,3 %

superadas com 24 h em otto  
(23º/3505)

# Robustez com mais tempo — e os custos que se pagam

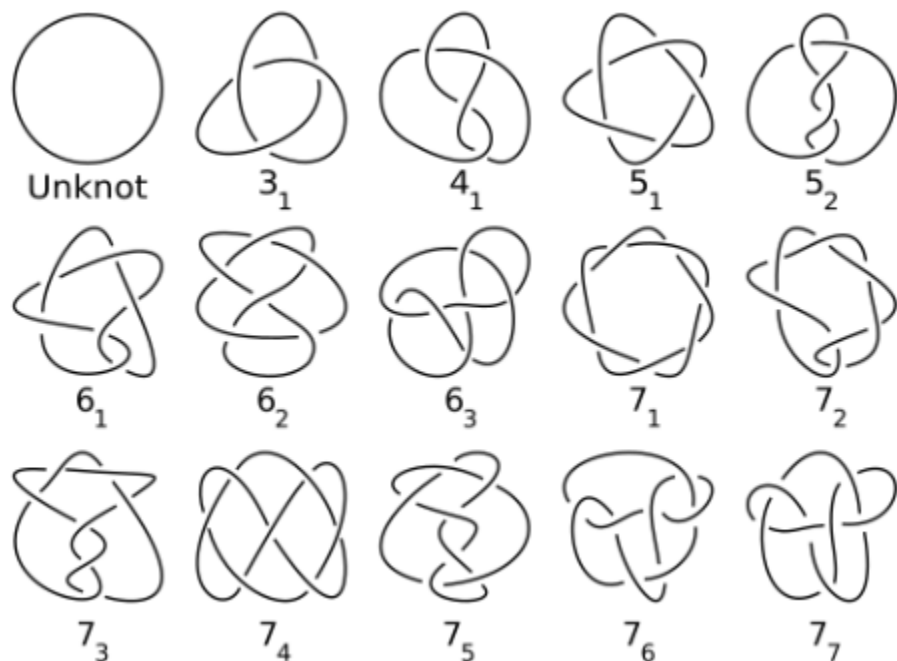
## Robustez

- 1 De 1 h para 4 h melhora em 30 dos 39 datasets; o auto-sklearn piora em 12 (overfitting).
- 2 Falhas em 4 h: AutoGluon 1 • TPOT 5 • auto-sklearn 6 • AutoWEKA 6 • H2O 8 • GCP 14.

## Limitações

- 1 Custo: mais tempo de treino e disco do que um modelo único bem ajustado.
- 2 Caixa-preta: centenas de modelos; há importância de atributos, não explicação simples.
- 3 Muitas classes: o stacking gera C atributos por modelo — Dionis (355 classes) estourou a memória.

# Implementação: Teoria de nós



```
from autogluon.tabular import TabularDataset, TabularPredictor
import matplotlib.pyplot as plt
```

```
# ---- 3) Carregar dados ----
```

```
data_url = 'https://raw.githubusercontent.com/mli/ag-docs/main/knot_theory/'
```

```
train_data = TabularDataset(f'{data_url}train.csv')
```

```
test_data = TabularDataset(f'{data_url}test.csv')
```

```
label = 'signature'
```

```
predictor = TabularPredictor(
```

```
    label=label,
```

```
    eval_metric='accuracy',
```

```
    path='AutogluonModels/knot_theory'
```

```
# métrica de otimização
```

```
# onde os modelos são salvos
```

```
).fit(
```

```
    train_data,
```

```
    time_limit=120,
```

```
    presets='medium_quality',
```

```
# <<< LIMITE DE TEMPO: 120 s (2 min)
```

```
# 'best_quality' = mais preciso porém mais lento
```

```
)
```

```
Label Column:      signature
```

```
AutoGluon infers your prediction problem is: 'multiclass' (because dtype is int64)
```

```
First 10 (of 13) unique label values: [np.int64(-2), np.int64(-1), np.int64(0), np.int64(1), np.int64(2), np.int64(3), np.int64(4), np.int64(5), np.int64(6), np.int64(7)]
```

```
If 'multiclass' is not the correct problem_type, please manually set problem_type to 'binary' or 'multiclass'.
```

```
Problem Type:      multiclass
```

```
Preprocessing data...
```

```
Warning: Some classes in the training set have fewer than 10 examples.
```

```
Fraction of data from classes with at least 10 examples that will be kept: 0.9230769230769231
```

```
Train Data Class Count: 9
```

```
AutoGluon training complete, total runtime = 121.19s ...
```

## Resultados: o ensemble em números reais

```
leaderboard = predictor.leaderboard(test_data)
# WeightedEnsemble_L2: score_test 0.9478, stack_level 2
# Ensemble Weights: NeuralNetFastAI 0.444, LightGBM 0.222,
#                      CatBoost 0.222, RandomForestEntr 0.111

metrics = predictor.evaluate(test_data)
# {'accuracy': 0.9478, 'balanced_accuracy': 0.7619, 'mcc': 0.936}

predictor.feature_importance(test_data)
# meridinal_translation_real: 0.52 (a mais importante)
```

|   | model               | score_test | score_val | eval_metric | pred_time_test | fit_order |
|---|---------------------|------------|-----------|-------------|----------------|-----------|
| 0 | WeightedEnsemble_L2 | 0.9478     | 0.961962  | accuracy    | 1.253661       | 7         |
| 1 | LightGBM            | 0.9456     | 0.955956  | accuracy    | 0.776162       | 3         |
| 2 | CatBoost            | 0.9432     | 0.955956  | accuracy    | 0.069079       | 6         |
| 3 | RandomForestEntr    | 0.9384     | 0.949950  | accuracy    | 0.300285       | 5         |
| 4 | RandomForestGini    | 0.9352     | 0.944945  | accuracy    | 0.333786       | 4         |
| 5 | LightGBMXT          | 0.9320     | 0.945946  | accuracy    | 1.305471       | 2         |
| 6 | NeuralNetFastAI     | 0.9304     | 0.946947  | accuracy    | 0.101000       | 1         |

## Conclusão: o que fica

O AutoGluon-Tabular é um AutoML de ponta a ponta para dados tabulares: dados crus entram, previsões saem, em uma linha de código.

Ele codifica as boas práticas da comunidade e as estende: stacking multicamadas com skip connections, repeated k-fold bagging e uma rede neural com embeddings e atalho linear.

Previsões out-of-fold em todas as camadas permitem um stacking profundo sem vazamento e usando 100% dos dados de treino.

A engenharia — treino sequencial, checkpoints, orçamento de tempo por camada — é o que garante robustez e tempo previsível em produção.

A evidência: campeão em 23 de 39 datasets do OpenML e em 7 de 11 competições Kaggle, superando 99% dos participantes em otto.

A lição maior: AutoML não é sinônimo de CASH. Combinar bem muitos modelos vale mais do que buscar o melhor — ao custo de tempo, disco e interpretabilidade.

# Referências

- 1 Erickson, N., Mueller, J., Shirkov, A., Zhang, H., Larroy, P., Li, M., Smola, A. (2020). AutoGluon-Tabular: Robust and Accurate AutoML for Structured Data. arXiv:2003.06505.
- 2 Caruana, R. et al. (2004). Ensemble selection from libraries of models. ICML.
- 3 Ting, K. M., Witten, I. H. (1997). Stacking bagged and dagged models. ICML.
- 4 Guo, C., Berkhahn, F. (2016). Entity embeddings of categorical variables. arXiv:1604.06737.
- 5 Gijsbers, P. et al. (2019). An open source AutoML benchmark. arXiv:1907.00909.
- 6 Documentação oficial: [auto.gluon.ai](https://auto.gluon.ai) — Tabular Quick Start. Benchmarks: [github.com/Innixa/autogluon-benchmarking](https://github.com/Innixa/autogluon-benchmarking).

# Quiz



<https://forms.gle/yXwaeATVarCn2Gq38>



# Obrigado!

Perguntas?

Bryan Fernando Sarango Rodriguez · [bryan.rodriguez@dtel.inatel.br](mailto:bryan.rodriguez@dtel.inatel.br)